



G&J Aircon Solutions - Technical Documentation

Version: 1.0

Last Updated: December 20, 2025

Technology Stack: Flutter, Dart, Supabase (PostgreSQL)



Table of Contents

- [1. Architecture Overview](#)
- [2. Data Layer](#)
- [3. Services Layer](#)
- [4. Database Operations](#)
- [5. Authentication Flow](#)
- [6. PDF Generation](#)
- [7. Real-time Features](#)

1. Architecture Overview

1.1 Project Structure

```
lib/
├─ main.dart           # App entry point & routing
├─ login.dart          # Authentication screen
├─ dashboard.dart      # Main dashboard
├─ scheduling.dart     # Job order management
├─ customers.dart      # Customer CRUD
├─ technicians.dart    # Technician CRUD
├─ aircons.dart        # Aircon unit registry
├─ expenses.dart       # Financial transactions
├─ payments.dart       # Payment records
├─ reports.dart        # Analytics & charts
├─ documents.dart      # Document management
├─ usermanage.dart     # User administration
├─ master_data.dart    # Reference data management
├─ service_items.dart  # Service catalog
├─ ui_app_shell.dart   # Main navigation shell
├─ shared_header.dart  # Reusable header component
├─
├─ data/
│   ├── app_state.dart # Global application state
│   ├── models.dart    # Data models/entities
│   ├── dashboard_provider.dart # Dashboard data provider
│   └─ activity_history.dart # Activity log viewer
├─
├─ services/
│   ├── activity_service.dart # Activity logging
│   ├── notification_service.dart # Push notifications
│   ├── pdf_generator.dart    # PDF document generation
│   └─ excel_generator.dart   # Excel report generation
├─
```

```
| shared/
|   └─ widgets.dart      # Reusable UI components
|
|   └─ theme/
|       └─ app_theme.dart  # Design system & tokens
```

1.2 Technology Stack

Component	Technology
Frontend Framework	Flutter 3.x
Programming Language	Dart
Backend/Database	Supabase (PostgreSQL)
Authentication	Supabase Auth
Real-time	Supabase Realtime
State Management	Provider + Local State
PDF Generation	pdf package
Excel Generation	excel package
Local Notifications	flutter_local_notifications
Environment Config	flutter_dotenv

2. Data Layer

2.1 Application State (app_state.dart)

Central state management for user session and global application state.

```
import 'package:flutter/material.dart';

enum UserRole { admin, serviceManager }

class AppState {
  static UserRole currentRole = UserRole.admin;
  static String currentUserName = 'Admin';
  static bool isSidebarCollapsed = false;

  // Shared job orders list - accessible by both admin and service manager
  static final List<dynamic> sharedJobOrders = [];
  static bool _jobOrdersSeeded = false;

  static bool get jobOrdersSeeded => _jobOrdersSeeded;
  static void setJobOrdersSeeded(bool value) => _jobOrdersSeeded = value;

  static bool hasShownWelcome = false;
```

```

static String headerWelcomeText() {
  final label = currentRole == UserRole.serviceManager
    ? 'Service Manager'
    : 'Admin';
  return 'Welcome, $label ($currentUserName)';
}

static String headerSubtitle() {
  return "";
}

static IconData roleAvatarIcon() {
  return currentRole == UserRole.serviceManager
    ? Icons.handyman_outlined
    : Icons.admin_panel_settings_outlined;
}

static String roleInitials() {
  if (currentUserName.isNotEmpty) {
    return currentUserName.trim()[0].toUpperCase();
  }
  return currentRole == UserRole.serviceManager ? 'S' : 'A';
}

static String roleDisplayName() {
  if (currentUserName.trim().isNotEmpty) {
    return currentUserName;
  }
  return currentRole == UserRole.serviceManager
    ? 'Service Manager'
    : 'Admin User';
}
}

```

2.2 Data Models (models.dart)

Complete data model definitions for the application entities.

```

enum CustomerTypeKind { b2b, b2c }

enum WorkTypeKind { installation, preventiveMaintenance, correctiveMaintenance }

enum JobOrderStatusKind { pending, inProgress, completed, cancelled }

enum PaymentMethodKind { cash, gcash, bankTransfer, card, cheque, other }

enum ExpenseCategoryKind { fuel, materials, food, transportation, toll, other }

class RoleData {
  final int id;
}

```

```
    final String role;
    const RoleData({required this.id, required this.role});
}

class UserData {
    final int id;
    final RoleData role;
    final String username;
    final String password;

    const UserData({
        required this.id,
        required this.role,
        required this.username,
        required this.password,
    });
}

class CustomerTypeData {
    final int id;
    final CustomerTypeKind type;
    const CustomerTypeData({required this.id, required this.type});
}

class CustomerData {
    final int id;
    final CustomerTypeData customerType;
    final String companyName;
    final String firstName;
    final String middleName;
    final String lastName;
    final String jobPosition;
    final String contactNumber;
    final String unitOrBuilding;
    final String street;
    final String subdivisionOrVillage;
    final String barangay;
    final String city;
    final String landmark;

    const CustomerData({
        required this.id,
        required this.customerType,
        required this.companyName,
        required this.firstName,
        required this.middleName,
        required this.lastName,
        required this.jobPosition,
        required this.contactNumber,
        required this.unitOrBuilding,
        required this.street,
        required this.subdivisionOrVillage,
```

```
        required this.barangay,
        required this.city,
        required this.landmark,
    });
}

class BrandData {
    final int id;
    final String name;
    const BrandData({required this.id, required this.name});
}

class AirconTypeData {
    final int id;
    final String typeName;
    const AirconTypeData({required this.id, required this.typeName});
}

class AirconData {
    final int id;
    final BrandData brand;
    final AirconTypeData airconType;
    final CustomerData customer;
    final String remarks;

    const AirconData({
        required this.id,
        required this.brand,
        required this.airconType,
        required this.customer,
        required this.remarks,
    });
}

class JobTypeData {
    final int id;
    final String jobType;
    const JobTypeData({required this.id, required this.jobType});
}

class JobStatusData {
    final int id;
    final String status;
    const JobStatusData({required this.id, required this.status});
}

class TechnicianData {
    final int id;
    final String firstName;
    final String middleName;
    final String lastName;
    final String contactNumber;
```

```

const TechnicianData({
  required this.id,
  required this.firstName,
  required this.middleName,
  required this.lastName,
  required this.contactNumber,
});
}

class JobOrderData {
  final int id;
  final JobTypeData jobType;
  final JobStatusData status;
  final UserData createdBy;
  final CustomerData customer;
  final String clientJoNumber;
  final DateTime dateCreated;
  final DateTime dateScheduled;
  final DateTime? dateCompleted;
  final CustomerTypeKind segment;
  final WorkTypeKind workType;

  const JobOrderData({
    required this.id,
    required this.jobType,
    required this.status,
    required this.createdBy,
    required this.customer,
    required this.clientJoNumber,
    required this.dateCreated,
    required this.dateScheduled,
    this.dateCompleted,
    required this.segment,
    required this.workType,
  });
}

class ServiceItemData {
  final int id;
  final String itemName;
  final String itemType;
  final double price;

  const ServiceItemData({
    required this.id,
    required this.itemName,
    required this.itemType,
    required this.price,
  });
}

```

```

class JobOrderLineItemData {
    final int id;
    final JobOrderData jobOrder;
    final ServiceItemData serviceItem;
    final int quantity;
    final double actualPrice;

    const JobOrderLineItemData({
        required this.id,
        required this.jobOrder,
        required this.serviceItem,
        required this.quantity,
        required this.actualPrice,
    });
}

class PaymentData {
    final int id;
    final JobOrderData jobOrder;
    final double amount;
    final DateTime paymentDate;
    final PaymentMethodKind paymentMethod;
    final String referenceNumber;
    final String orNumber;
    final String status;
    final String proofImageUrl;

    const PaymentData({
        required this.id,
        required this.jobOrder,
        required this.amount,
        required this.paymentDate,
        required this.paymentMethod,
        required this.referenceNumber,
        required this.orNumber,
        required this.status,
        required this.proofImageUrl,
    });
}

class ExpenseData {
    final int id;
    final JobOrderData? jobOrder;
    final double amount;
    final DateTime paymentDate;
    final PaymentMethodKind paymentMethod;
    final String referenceNumber;
    final String orNumber;
    final String status;
    final String proofImageUrl;
    final ExpenseCategoryKind category;
    final bool isCustomerFunded;
}

```

```
const ExpenseData({
  required this.id,
  this.jobOrder,
  required this.amount,
  required this.paymentDate,
  required this.paymentMethod,
  required this.referenceNumber,
  required this.orNumber,
  required this.status,
  required this.proofImageUrl,
  required this.category,
  required this.isCustomerFunded,
});
}
```

3. Services Layer

3.1 Activity Logger (activity_service.dart)

Logs user actions to the database for audit trail.

```
import 'package:supabase_flutter/supabase_flutter.dart';

class ActivityLogger {
  static final _supabase = Supabase.instance.client;

  /// Call this whenever a user does something important
  static Future<void> log({
    required String type, // e.g., 'Create', 'Delete', 'Payment'
    required String details, // e.g., 'Created Job JO-1234'
  }) async {
    try {
      final user = _supabase.auth.currentUser;
      if (user == null) return;

      // Get the current user's name
      final userDetails = await _supabase
        .from('app_users')
        .select('full_name')
        .eq('id', user.id)
        .maybeSingle();

      final name = userDetails?['full_name'] ?? user.email ?? 'Unknown User';

      // Insert the log
      await _supabase.from('activity_logs').insert({
        'user_id': user.id,
        'user_name': name,
        'action_type': type,
        'details': details,
      });
    } catch (e) {
      // Log the error
    }
  }
}
```



```

        'created_at': DateTime.now().toIso8601String(),
    });
} catch (e) {
    // Fail silently so we don't crash the app if logging fails
    print('Failed to log activity: $e');
}
}
}

```

3.2 Notification Service (notification_service.dart)

Handles local push notifications with platform-specific configurations.

```

import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:shared_preferences/shared_preferences.dart';

class NotificationService {
    static final NotificationService _instance = NotificationService._internal();
    factory NotificationService() => _instance;
    NotificationService._internal();

    final FlutterLocalNotificationsPlugin _notificationsPlugin =
        FlutterLocalNotificationsPlugin();
    bool _isEnabled = false;

    bool get isEnabled => _isEnabled;

    Future<void> init() async {
        // Load preference
        final prefs = await SharedPreferences.getInstance();
        _isEnabled = prefs.getBool('notifications_enabled') ?? false;

        // Android Init
        const androidSettings = AndroidInitializationSettings('@mipmap/ic_launcher');

        // iOS/macOS Init
        final darwinSettings = const DarwinInitializationSettings(
            requestAlertPermission: false,
            requestBadgePermission: false,
            requestSoundPermission: false,
        );

        final initSettings = InitializationSettings(
            android: androidSettings,
            iOS: darwinSettings,
            macOS: darwinSettings,
        );

        await _notificationsPlugin.initialize(initSettings);
    }
}

```

```

Future<void> toggleNotifications(bool value) async {
  _isEnabled = value;
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('notifications_enabled', value);

  if (value) {
    await _requestPermissions();
  }
}

Future<void> _requestPermissions() async {
  await _notificationsPlugin.resolvePlatformSpecificImplementation<
    AndroidFlutterLocalNotificationsPlugin>()?.requestNotificationsPermission();
  await _notificationsPlugin.resolvePlatformSpecificImplementation<
    IOSFlutterLocalNotificationsPlugin>()?.requestPermissions(
    alert: true, badge: true, sound: true);
  await _notificationsPlugin.resolvePlatformSpecificImplementation<
    MacOSFlutterLocalNotificationsPlugin>()?.requestPermissions(
    alert: true, badge: true, sound: true);
}

Future<void> showNotification({
  required int id,
  required String title,
  required String body
}) async {
  if (!_isEnabled) return;

  final BigTextStyleInformation bigTextStyleInformation = BigTextStyleInformation(
    body,
    htmlFormatBigText: true,
    contentType: title,
    htmlFormatContentType: true,
  );

  final AndroidNotificationDetails androidDetails = AndroidNotificationDetails(
    'high_importance_channel',
    'High Importance Notifications',
    importance: Importance.max,
    priority: Priority.high,
    styleInformation: bigTextStyleInformation,
  );

  final notificationDetails = NotificationDetails(
    android: androidDetails,
    iOS: DarwinNotificationDetails(
      presentAlert: true, presentBadge: true, presentSound: true),
    macOS: DarwinNotificationDetails(
      presentAlert: true, presentBadge: true, presentSound: true),
  );

  await _notificationsPlugin.show(id, title, body, notificationDetails);
}

```

```
}  
}
```

4. Database Operations

4.1 Dashboard Provider (dashboard_provider.dart)

Central provider for dashboard data with dynamic date range filtering.

```
import 'package:flutter/material.dart';  
import 'package:supabase_flutter/supabase_flutter.dart';  
import '../services/notification_service.dart';  
  
class DashboardProvider extends ChangeNotifier {  
  static final DashboardProvider _instance = DashboardProvider._internal();  
  factory DashboardProvider() => _instance;  
  DashboardProvider._internal();  
  
  // Cache variables  
  int _pendingJobsCount = 0;  
  int _pendingDocsCount = 0;  
  double _totalRevenue = 0.0;  
  double _totalExpenses = 0.0;  
  List<TodayJobItem> _todayJobs = [];  
  List<AttentionItem> _notificationItems = [];  
  List<AttentionItem> _activityItems = [];  
  DateTime? _lastFetch;  
  
  // Getters  
  int get pendingJobsCount => _pendingJobsCount;  
  int get pendingDocsCount => _pendingDocsCount;  
  double get totalRevenue => _totalRevenue;  
  double get totalExpenses => _totalExpenses;  
  List<TodayJobItem> get todayJobs => _todayJobs;  
  List<AttentionItem> get notificationItems => _notificationItems;  
  List<AttentionItem> get activityItems => _activityItems;  
  
  // --- MAIN FETCH METHOD ---  
  Future<void> fetchDashboardData({String dateRange = 'Monthly'}) async {  
    try {  
      await Future.wait([  
        _fetchPendingJobs(dateRange),  
        _fetchScheduledJobs(dateRange),  
        _fetchTotalRevenue(dateRange),  
        _fetchTotalExpenses(dateRange),  
        _fetchNotifications(),  
        _fetchActivityLogs(),  
      ]);  
      _lastFetch = DateTime.now();  
      notifyListeners();  
    } catch (e) {
```

```

        debugPrint('Error fetching dashboard data: $e');
    }
}

// --- HELPER: Date Ranges ---
List<String> _getDateRange(String range) {
    final now = DateTime.now();
    DateTime start;
    DateTime end;

    switch (range) {
        case 'Weekly':
            start = now.subtract(Duration(days: now.weekday - 1));
            end = start.add(const Duration(days: 6));
            break;
        case 'Monthly':
            start = DateTime(now.year, now.month, 1);
            end = DateTime(now.year, now.month + 1, 0);
            break;
        case 'Yearly':
            start = DateTime(now.year, 1, 1);
            end = DateTime(now.year, 12, 31);
            break;
        case 'All':
            start = DateTime(2000, 1, 1);
            end = DateTime(2100, 12, 31);
            break;
        case 'Today':
        default:
            start = now;
            end = now;
            break;
    }

    String fmt(DateTime d) =>
        "${d.year}-${d.month.toString().padLeft(2, '0')}-${d.day.toString().padLeft(2, '0')}";

    return [fmt(start), fmt(end)];
}

// --- PENDING JOBS ---
Future<void> _fetchPendingJobs(String dateRange) async {
    try {
        final supabase = Supabase.instance.client;
        final range = _getDateRange(dateRange);

        final response = await supabase
            .from('job_orders')
            .select('id')
            .inFilter('status', ['Pending', 'Scheduled', 'In Progress'])
            .gte('date_scheduled', "${range[0]}T00:00:00")
    }
}

```

```

        .lte('date_scheduled', "${range[1]}T23:59:59");

    _pendingJobsCount = response.length;
} catch (e) {
    debugPrint('Error fetching pending jobs: $e');
    _pendingJobsCount = 0;
}
}

// --- REVENUE CALCULATION ---
Future<void> _fetchTotalRevenue(String dateRange) async {
    try {
        final supabase = Supabase.instance.client;
        final range = _getDateRange(dateRange);

        // Job Payments (Verified)
        final payments = await supabase
            .from('payments')
            .select('amount')
            .eq('status', 'Verified')
            .gte('payment_date', range[0])
            .lte('payment_date', range[1]);

        double total = payments.fold(
            0.0,
            (sum, row) => sum + (row['amount'] ?? 0.0),
        );

        // Manual Revenue (Other Income)
        final otherIncome = await supabase
            .from('expenses')
            .select('amount')
            .eq('is_income', true)
            .neq('expense_type', 'Added Funds')
            .neq('expense_type', 'Capital')
            .neq('expense_type', 'Personal')
            .gte('date', range[0])
            .lte('date', range[1]);

        total += otherIncome.fold(
            0.0,
            (sum, row) => sum + (row['amount'] ?? 0.0),
        );

        _totalRevenue = total;
    } catch (e) {
        debugPrint('Error fetching revenue: $e');
        _totalRevenue = 0.0;
    }
}

// --- EXPENSES CALCULATION ---

```

```

Future<void> _fetchTotalExpenses(String dateRange) async {
  try {
    final supabase = Supabase.instance.client;
    final range = _getDateRange(dateRange);

    final expenses = await supabase
      .from('expenses')
      .select('amount')
      .neq('is_income', true)
      .neq('expense_type', 'Personal')
      .gte('date', range[0])
      .lte('date', range[1]);

    _totalExpenses = expenses.fold(
      0.0,
      (sum, row) => sum + (row['amount'] ?? 0.0),
    );
  } catch (e) {
    debugPrint('Error fetching expenses: $e');
    _totalExpenses = 0.0;
  }
}

// ... Additional methods for notifications and activity logs
}

// Data models for dashboard
class TodayJobItem {
  final String id;
  final String client;
  final String time;
  final String date;
  final String type;
  final String status;

  TodayJobItem({
    required this.id,
    required this.client,
    required this.time,
    required this.date,
    required this.type,
    required this.status,
  });
}

enum AttentionType { payment, expense, scheduling, document }

class AttentionItem {
  final String title;
  final String reference;
  final String priority;
  final Color color;
}

```

```

final AttentionType type;
final int? relatedId;
final String? searchContext;
final String? customerName;
final String? serviceType;
final String? date;

AttentionItem({
  required this.title,
  required this.reference,
  required this.priority,
  required this.color,
  required this.type,
  this.relatedId,
  this.searchContext,
  this.customerName,
  this.serviceType,
  this.date,
});
}

```

4.2 Job Order CRUD Operations

Key database operations for job order management from `scheduling.dart` .

Fetching Job Orders

```

Future<void> _fetchJobOrders() async {
  setState(() => _isLoading = true);
  final supabase = Supabase.instance.client;

  try {
    final response = await supabase
      .from('job_orders')
      .select('''
        *,
        customers(id, first_name, last_name, company_name, city, barangay,
          address_complete, customer_type_id),
        job_types(job_type_name),
        job_order_line_items(quantity, actual_price),
        payments(amount),
        job_order_technicians(count),
        job_order_aircons(count)
      ''')
      .order('date_scheduled', ascending: false);

    final List<JobOrder> loaded = [];

    for (var row in response) {
      // Parse customer data
      final cust = row['customers'];
    }
  }
}

```

```

String clientName = 'Unknown';
if (cust != null) {
    clientName = cust['company_name'] ??
        '${cust['first_name']} ${cust['last_name']}'?.trim();
}

// Calculate billing totals
final lineItems = row['job_order_line_items'] as List? ?? [];
final payments = row['payments'] as List? ?? [];

double billed = lineItems.fold(
    0.0,
    (s, i) => s + ((i['quantity'] ?? 1) * (i['actual_price'] ?? 0.0))
);
double paid = payments.fold(
    0.0,
    (s, p) => s + (p['amount'] ?? 0.0)
);

bool unbilled = billed == 0;
bool unpaid = billed > 0 && paid < billed;

// Build JobOrder object
loaded.add(JobOrder(
    dbId: row['id'],
    displayId: row['client_jo_number'] ?? 'JO-${row['id']}',
    clientName: clientName,
    startDateTime: DateTime.parse(row['date_scheduled']),
    status: row['status'] ?? 'Pending',
    isUnbilled: unbilled,
    isUnpaid: unpaid,
    // ... other properties
));
}

if (mounted) setState(() => _orders = loaded);
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Error loading jobs: $e'))
        );
    }
} finally {
    if (mounted) setState(() => _isLoading = false);
}
}

```

Creating a Job Order

```

Future<void> _submit() async {
    final scheduleDateTime = DateTime(

```



```

        _scheduleDate.year,
        _scheduleDate.month,
        _scheduleDate.day,
        _scheduleTime.hour,
        _scheduleTime.minute,
    );

    // Validation: Past time check
    if (widget.existingJob == null &&
        scheduleDateTime.isBefore(DateTime.now().subtract(const Duration(minutes: 1)))) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Cannot schedule a job in the past.')),
        );
        return;
    }

    setState(() => _isSubmitting = true);

    try {
        int customerId;

        // Create new customer if needed
        if (_isNewClient) {
            final customerTypeId = _customerType == 'Commercial' ? 2 : 1;

            final newCustomer = await _supabase
                .from('customers')
                .insert({
                    'first_name': _firstNameController.text.trim(),
                    'last_name': _lastNameController.text.trim(),
                    'company_name': _companyController.text.trim(),
                    'contact_number': _contactController.text.trim(),
                    'address_complete': _addressController.text.trim(),
                    'customer_type_id': customerTypeId,
                })
                .select('id')
                .single();

            customerId = newCustomer['id'];
        } else {
            customerId = _selectedClientId!;
        }

        // Create the job order
        final jobData = {
            'customer_id': customerId,
            'job_type_id': _jobTypeId,
            'status': 'Pending',
            'date_scheduled': scheduleDateTime.toUtc().toIso8601String(),
            'notes': _notesController.text.trim(),
            'client_jo_number': _externalRefController.text.trim(),
        };
    }

```

```

final res = await _supabase
    .from('job_orders')
    .insert(jobData)
    .select('id')
    .single();

final newJobId = res['id'];

// Link technicians
if (_selectedTechnicianIds.isNotEmpty) {
    final techLinks = _selectedTechnicianIds
        .map((tid) => {'job_order_id': newJobId, 'technician_id': tid})
        .toList();
    await _supabase.from('job_order_technicians').insert(techLinks);
}

// Link aircon units
if (_selectedAirconIds.isNotEmpty) {
    final unitLinks = _selectedAirconIds
        .map((aid) => {'job_order_id': newJobId, 'aircon_id': aid})
        .toList();
    await _supabase.from('job_order_aircons').insert(unitLinks);
}

// Log the activity
await ActivityLogger.log(
    type: 'Create',
    details: 'Created Job Order ${_externalRefController.text}',
);

if (mounted) {
    Navigator.pop(context);
    widget.onCreated?.call();
}
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Error: $e')),
        );
    }
} finally {
    if (mounted) setState(() => _isSubmitting = false);
}
}

```

Deleting a Job Order

```

void _deleteJob() async {
    final confirm = await showConfirmDialog(
        context: context,

```

```

        title: "Delete Job?",
        message: "This will remove the job and all billing records.",
        confirmLabel: "Delete Forever",
        isDestructive: true,
    );

    if (confirm == true) {
      try {
        await _supabase.from('job_orders').delete().eq('id', widget.job.dbId);

        // Log the deletion
        await ActivityLogger.log(
          type: 'Delete',
          details: 'Deleted Job ${widget.job.displayId}',
        );

        if (mounted) {
          Navigator.pop(context);
          widget.onJobUpdated();
        }
      } catch (e) {
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text("Delete Error: $e")),
          );
        }
      }
    }
  }
}

```

Recording a Payment

```

void _recordPaymentDialog() {
  final balance = _totalAmount - _totalPaid;
  final amountController = TextEditingController(
    text: balance > 0 ? balance.toStringAsFixed(2) : '',
  );
  final refController = TextEditingController();
  final orController = TextEditingController();
  String method = 'Cash';

  showDialog(
    context: context,
    builder: (context) => StatefulBuilder(
      builder: (context, setState) {
        return AlertDialog(
          title: const Text("Record Payment"),
          content: Form(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [

```

```

TextFormField(
  controller: amountController,
  decoration: const InputDecoration(labelText: "Amount Received"),
  keyboardType: TextInputType.number,
),
DropdownButtonFormField<String>(
  value: method,
  items: ['Cash', 'GCash', 'Bank Transfer', 'Card', 'Cheque']
    .map((m) => DropdownMenuItem(value: m, child: Text(m)))
    .toList(),
  onChanged: (v) => setState(() => method = v!),
  decoration: const InputDecoration(labelText: "Payment Method"),
),
if (method != 'Cash')
  TextFormField(
    controller: refController,
    decoration: const InputDecoration(labelText: "Reference No."),
  ),
],
),
),
actions: [
  TextButton(
    onPressed: () => Navigator.pop(context),
    child: const Text("Cancel"),
  ),
  ElevatedButton(
    onPressed: () async {
      final amount = double.tryParse(amountController.text) ?? 0;
      if (amount <= 0) return;

      await _supabase.from('payments').insert({
        'job_order_id': widget.job.dbId,
        'amount': amount,
        'payment_method': method,
        'reference_number': refController.text,
        'or_number': orController.text,
        'payment_date': DateTime.now().toIso8601String(),
        'status': method == 'Cash' ? 'Verified' : 'Pending',
      });

      await ActivityLogger.log(
        type: 'Payment',
        details: 'Recorded PHP $amount for ${widget.job.displayId}',
      );

      Navigator.pop(context);
      _fetchBillingData();
    },
    child: const Text("Save Payment"),
  ),
],

```

```
    );  
  },  
),  
);  
}
```

5. Authentication Flow

5.1 Application Entry (`main.dart`)

```
import 'package:flutter/material.dart';  
import 'package:supabase_flutter/supabase_flutter.dart';  
import 'package:flutter_dotenv/flutter_dotenv.dart';  
import 'data/app_state.dart';  
import 'services/notification_service.dart';  
  
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  // Load Environment Variables  
  await dotenv.load(fileName: ".env");  
  
  // Initialize Supabase  
  await Supabase.initialize(  
    url: dotenv.env['SUPABASE_URL']!,  
    anonKey: dotenv.env['SUPABASE_ANON_KEY']!,  
  );  
  
  // Initialize Notifications  
  await NotificationService().init();  
  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'G&J Aircon Solutions',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF2563EB)),  
        useMaterial3: true,  
        scaffoldBackgroundColor: const Color(0xFFF3F6FB),  
        fontFamily: 'Roboto',  
      ),  
      home: const AuthInitializer(),  
      routes: {
```

```

    '/login': (_) => const LoginScreen(),
    '/dashboard': (_) => const DashboardScreen(),
    '/scheduling': (_) => const SchedulingScreen(),
    '/expenses': (_) => const ExpensesScreen(),
    '/payments': (_) => const PaymentsScreen(),
    '/documents': (_) => const DocumentsScreen(),
    '/reports': (_) => const ReportsScreen(),
    '/customers': (_) => const CustomersScreen(),
    '/technicians': (_) => const TechniciansScreen(),
    '/aircons': (_) => const AirconsScreen(),
    '/service-items': (_) => const ServiceItemsScreen(),
    '/master-data': (_) => const MasterDataScreen(),
    '/users': (_) => const UserManagementScreen(),
    '/settings': (_) => const SettingsScreen(),
  },
);
}
}

```

5.2 Session Resolver

```

class AuthInitializer extends StatefulWidget {
  const AuthInitializer({super.key});

  @override
  State<AuthInitializer> createState() => _AuthInitializerState();
}

class _AuthInitializerState extends State<AuthInitializer> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _resolveUserSession();
    });
  }

  Future<void> _resolveUserSession() async {
    final supabase = Supabase.instance.client;
    final session = supabase.auth.currentSession;

    // No Session? -> Go to Login
    if (session == null) {
      _goToLogin();
      return;
    }

    // Validate Email
    final userEmail = session.user.email;
    if (userEmail == null) {
      await supabase.auth.signOut();
    }
  }
}

```

```

        _goToLogin();
        return;
    }

    // Query DB with TIMEOUT
    try {
        final data = await supabase
            .from('app_users')
            .select('*', roles(role_name))
            .eq('email', userEmail)
            .maybeSingle()
            .timeout(const Duration(seconds: 10));

        if (data == null) {
            await supabase.auth.signOut();
            _goToLogin();
            return;
        }

        // Update AppState
        final roleData = data['roles'];
        final roleName = roleData != null ? roleData['role_name'] : 'User';
        final roleString = roleName.toString().toLowerCase();

        if (roleString == 'service manager') {
            AppState.currentRole = UserRole.serviceManager;
        } else {
            AppState.currentRole = UserRole.admin;
        }

        final fullName = data['full_name'];
        if (fullName != null) {
            AppState.currentUserName = fullName;
        }

        // Navigate to Dashboard
        if (mounted) {
            Navigator.of(context).pushReplacementNamed('/dashboard');
        }
    } catch (e) {
        debugPrint("AuthInit CRITICAL ERROR: $e");
        await supabase.auth.signOut();
        _goToLogin();
    }
}

void _goToLogin() {
    if (mounted) {
        Navigator.of(context).pushReplacementNamed('/login');
    }
}

```

```

@override
Widget build(BuildContext context) {
  return const Scaffold(
    backgroundColor: Colors.white,
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          CircularProgressIndicator(),
          SizedBox(height: 20),
          Text("Loading G&J System..."),
        ],
      ),
    ),
  );
}

```

6. PDF Generation

6.1 Statement of Account (SOA)

```

import 'package:flutter/services.dart';
import 'package:pdf/pdf.dart';
import 'package:pdf/widgets.dart' as pw;
import 'package:intl/intl.dart';

class SOAData {
  final String customerName;
  final String customerAddress;
  final String soaNumber;
  final DateTime soaDate;
  final List<SOAItem> items;

  SOAData({
    required this.customerName,
    required this.customerAddress,
    required this.soaNumber,
    required this.soaDate,
    required this.items,
  });

  double get total => items.fold(0, (sum, item) => sum + item.total);
}

class SOAItem {
  final String clientName;
  final String workDescription;
  final double amount;
}

```



```

SOAItem({
  required this.clientName,
  required this.workDescription,
  required this.amount,
});

double get total => amount;
}

class PDFGeneratorService {
  static final dateFormat = DateFormat('MMM dd, yyyy');

  static String formatCurrency(double amount) {
    final formatter = NumberFormat('#,##0.00', 'en_US');
    return 'PHP ${formatter.format(amount)}';
  }

  static Future<Uint8List> generateSOA(SOAData data) async {
    final pdf = pw.Document();

    // Load logo
    final logoData = await rootBundle.load('lib/image/logo.png');
    final logoImage = pw.MemoryImage(logoData.buffer.asUint8List());

    pdf.addPage(
      pw.Page(
        pageFormat: PdfPageFormat.a4,
        margin: const pw.EdgeInsets.all(40),
        build: (pw.Context context) {
          return pw.Column(
            crossAxisAlignment: pw.CrossAxisAlignment.start,
            children: [
              // Header with company info
              _buildHeader(logoImage),
              pw.SizedBox(height: 20),

              // SOA Title
              pw.Center(
                child: pw.Column(
                  children: [
                    pw.Text(
                      'Statement of Accounts',
                      style: pw.TextStyle(
                        fontSize: 20,
                        fontWeight: pw.FontWeight.bold,
                      ),
                    ),
                    pw.SizedBox(height: 5),
                    pw.Text(
                      'Reference: ${data.soaNumber}',
                      style: const pw.TextStyle(fontSize: 11),
                    ),
                  ],
                ),
              ),
            ],
          );
        },
      ),
    );
  }
}

```

```

        ],
    ),
),
pw.SizedBox(height: 20),

// Bill To section
pw.Column(
  crossAxisAlignment: pw.CrossAxisAlignment.start,
  children: [
    pw.Text('Bill to:', style: const pw.TextStyle(fontSize: 10)),
    pw.SizedBox(height: 3),
    pw.Text(data.customerName,
      style: pw.TextStyle(fontSize: 11, fontWeight: pw.FontWeight.bold)),
    pw.SizedBox(height: 8),
    pw.Text('Address:', style: const pw.TextStyle(fontSize: 10)),
    pw.SizedBox(height: 3),
    pw.Text(data.customerAddress, style: const pw.TextStyle(fontSize: 11)),
  ],
),
pw.SizedBox(height: 20),

// Date aligned right
pw.Align(
  alignment: pw.Alignment.centerRight,
  child: pw.Text('Date: ${dateFormat.format(data.soaDate)}',
    style: const pw.TextStyle(fontSize: 10)),
),
pw.SizedBox(height: 15),

// Items Table
_buildItemsTable(data.items),

// Totals
_buildTotalsSection(data),
pw.SizedBox(height: 25),

// Payment Instructions
pw.Text(
  'Please send payment of the total amount to the bank account below:',
  style: pw.TextStyle(fontSize: 10, fontWeight: pw.FontWeight.bold),
),
pw.SizedBox(height: 5),
pw.Text('Bank: Bank of the Philippine Islands (BPI)',
  style: const pw.TextStyle(fontSize: 9)),
pw.Text('Account Number: 2149-7202-41',
  style: const pw.TextStyle(fontSize: 9)),
pw.Text('Account Name: Jemima Obsequio',
  style: const pw.TextStyle(fontSize: 9)),
],
);
},
),

```

```

    );

    return pdf.save();
}

static pw.Widget _buildHeader(pw.MemoryImage logo) {
    return pw.Center(
        child: pw.Column(
            children: [
                pw.Image(logo, width: 80, height: 80),
                pw.SizedBox(height: 8),
                pw.Text(
                    "G AND J Aircon Solutions, Door 9, Teresita's Promenade, De Guzman St.",
                    style: const pw.TextStyle(fontSize: 9),
                ),
                pw.Text(
                    'Toril Proper, Davao City, 8000',
                    style: const pw.TextStyle(fontSize: 9),
                ),
            ],
        ),
    );
}

static pw.Widget _buildItemsTable(List<SOAItem> items) {
    return pw.Table(
        border: pw.TableBorder.all(color: PdfColors.grey800, width: 1),
        children: [
            // Header
            pw.TableRow(
                children: [
                    _buildTableHeader('CLIENT'),
                    _buildTableHeader('WORK'),
                    _buildTableHeader('AMOUNT', align: pw.TextAlign.right),
                ],
            ),
            // Items
            ...items.map((item) => pw.TableRow(
                children: [
                    _buildTableCell(item.clientName),
                    _buildTableCell(item.workDescription),
                    _buildTableCell(formatCurrency(item.amount), align: pw.TextAlign.right),
                ],
            )),
        ],
    );
}

static pw.Widget _buildTotalsSection(SOADATA data) {
    return pw.Container(
        decoration: pw.BoxDecoration(
            border: pw.Border.all(color: PdfColors.grey800, width: 1),

```

```

    ),
    child: pw.Row(
      children: [
        pw.Expanded(
          child: pw.Container(
            padding: const pw.EdgeInsets.all(8),
            alignment: pw.Alignment.centerRight,
            child: pw.Text('TOTAL',
              style: pw.TextStyle(fontSize: 12, fontWeight: pw.FontWeight.bold)),
          ),
        ),
        pw.Container(
          width: 150,
          padding: const pw.EdgeInsets.all(8),
          decoration: pw.BoxDecoration(
            border: pw.Border(left: pw.BorderSide(color: PdfColors.grey800, width: 1)),
          ),
          alignment: pw.Alignment.centerRight,
          child: pw.Text(
            formatCurrency(data.total),
            style: pw.TextStyle(fontSize: 12, fontWeight: pw.FontWeight.bold),
          ),
        ),
      ],
    ),
  );
}

```

6.2 Excel Report Generation

```

import 'dart:typed_data';
import 'package:flutter/services.dart' show rootBundle;
import 'package:excel/excel.dart';
import 'package:intl/intl.dart';

class WeeklyReportData {
  final String item;
  final String bank;
  final String branch;
  final String branchCode;
  final String sr;
  final DateTime dateReported;
  final DateTime dateCompleted;
  final String description;
  final String status;
  final String trade;

  WeeklyReportData({
    required this.item,
    required this.bank,

```

```

        required this.branch,
        required this.branchCode,
        required this.sr,
        required this.dateReported,
        required this.dateCompleted,
        required this.description,
        required this.status,
        required this.trade,
    });
}

class ExcelGeneratorService {
    static const String _templatePath = 'lib/templatedocu/zWeekly Report - TEMPLATE.xlsx';

    static Future<Uint8List?> generateWeeklyReport(WeeklyReportData data) async {
        try {
            final ByteData templateData = await rootBundle.load(_templatePath);
            final List<int> bytes = templateData.buffer.asUint8List();
            final Excel excel = Excel.decodeBytes(bytes);

            final String sheetName = excel.tables.keys.first;
            final Sheet sheet = excel[sheetName];

            final dateFormat = DateFormat('MM/dd/yyyy');

            final List<CellValue> rowData = [
                TextCellValue(data.item),
                TextCellValue(data.bank),
                TextCellValue(data.branch),
                TextCellValue(data.branchCode),
                TextCellValue(data.sr),
                TextCellValue(dateFormat.format(data.dateReported)),
                TextCellValue(dateFormat.format(data.dateCompleted)),
                TextCellValue(data.description),
                TextCellValue(data.status),
                TextCellValue(data.trade),
            ];

            sheet.appendRow(rowData);

            return Uint8List.fromList(excel.encode());
        } catch (e) {
            print('Error generating Excel: $e');
            return null;
        }
    }
}

```

7. Real-time Features

7.1 Realtime Subscription Setup

From `dashboard.dart` :

```
void _setupRealtimeSubscription() {
  final supabase = Supabase.instance.client;

  // Subscribe to job_orders changes
  _subscription = supabase
    .channel('dashboard_updates')
    .onPostgresChanges(
      event: PostgresChangeEvent.all,
      schema: 'public',
      table: 'job_orders',
      callback: (payload) {
        debugPrint('Realtime update received: ${payload.eventType}');
        _refreshData();
      },
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.all,
      schema: 'public',
      table: 'payments',
      callback: (payload) {
        debugPrint('Payment update received: ${payload.eventType}');
        _refreshData();
      },
    )
    .subscribe();
}

@override
void dispose() {
  _subscription?.unsubscribe();
  super.dispose();
}
```

Database Schema Reference

Core Tables

Table	Description
app_users	User accounts with role references
roles	Role definitions (Admin, Service Manager)
customers	Customer records
technicians	Technician records

brands	Aircon brand master data
aircon_types	Aircon type master data
aircons	Customer aircon units
job_types	Job type definitions
job_orders	Main job order records
job_order_line_items	Service items per job
job_order_technicians	Technician assignments
job_order_aircons	Aircon unit assignments
service_items	Service catalog
payments	Payment records
expenses	Expense transactions
activity_logs	User activity audit trail
documents	Generated document metadata

Environment Variables

Required `.env` configuration:

```
SUPABASE_URL=your_supabase_project_url
SUPABASE_ANON_KEY=your_supabase_anon_key
```